

Unidad V: Árboles

Clase 9 – Resolver problemas con ABB

Objetivo

Aplicar ABB a problemas concretos y estimar costo.

Tema

- Altura, nodos hoja, balance conceptual. Aplicaciones prácticas.

Introducción

En la clase anterior se presentó el árbol como estructura jerárquica, se distinguió entre árbol general, árbol binario y Árbol Binario de Búsqueda, y se trabajó con la construcción manual de un ABB a partir de una secuencia de inserciones. También se introdujo la idea de recursividad como una forma natural de resolver problemas sobre árboles, porque cada subárbol puede ser tratado como un árbol más pequeño.

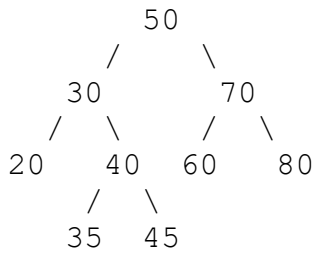
En esta clase vamos a avanzar un paso más: ya no alcanza con saber construir el árbol, sino que necesitamos **resolver problemas sobre él**. Para eso vamos a trabajar con métodos que calculan información, buscan elementos, cuentan nodos, identifican hojas, determinan alturas, listan valores bajo una condición y detectan situaciones de desbalance.

El objetivo central es que puedan mirar un problema y diseñar una solución modular antes de programarla. En ABB, casi todo ejercicio se puede pensar con tres preguntas: **qué hago si el árbol está vacío, qué hago con la raíz y qué hago con el subárbol izquierdo y el subárbol derecho**. Esta forma de razonamiento es clave porque el apunte de cátedra define el árbol como una raíz con dos subárboles, y por eso la mayoría de sus operaciones se apoyan en recursividad.

1. Repaso: qué es un ABB

Un **Árbol Binario de Búsqueda**, o **ABB**, es un árbol binario que cumple una regla de orden. Para cada nodo del árbol, todos los valores del subárbol izquierdo son menores que el valor del nodo, y todos los valores del subárbol derecho son mayores. Además, en el enfoque trabajado por la cátedra, no se admiten elementos repetidos.

Por ejemplo:



Este árbol es un ABB porque respeta la regla de orden en cada nodo. A la izquierda de 50 están los valores menores; a la derecha, los mayores. Lo mismo ocurre con 30, con 40 y con 70.

La propiedad de orden permite hacer búsquedas más eficientes que en una lista, porque no siempre es necesario recorrer todos los nodos. Si buscamos un valor menor que la raíz, no tiene sentido ir al subárbol derecho. Si buscamos un valor mayor, no tiene sentido ir al subárbol izquierdo.

2. La forma general de resolver problemas con ABB

La mayoría de los problemas con ABB se resuelven con una estructura recursiva. El árbol puede estar vacío o puede tener una raíz con dos subárboles. Por eso, antes de escribir código, conviene pensar la solución con este molde:

```

public static tipo resolver(ABBTDA a) {
    if (a.ArbolVacio()) {
        // caso base
    } else {
        // proceso la raíz
        // resuelvo el problema en el hijo izquierdo
        // resuelvo el problema en el hijo derecho
    }
}
  
```

La parte más importante es el **caso base**. Si el árbol está vacío, el método debe saber qué devolver o qué hacer. Por ejemplo, si estamos contando nodos, un árbol vacío tiene 0 nodos. Si estamos sumando valores, un árbol vacío aporta 0. Si estamos buscando un valor, un árbol vacío significa que el valor no fue encontrado.

Luego viene el **paso recursivo**. Si el árbol no está vacío, se analiza la raíz y después se decide qué hacer con los subárboles. En algunos problemas se recorren ambos hijos; en otros, como la búsqueda, se aprovecha la regla del ABB para bajar solo por un camino.

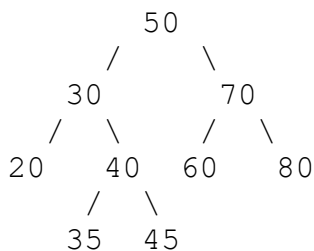
3. Contar la cantidad total de nodos

El primer problema clásico es contar cuántos nodos tiene el árbol.

La idea es simple: si el árbol está vacío, tiene 0 nodos. Si no está vacío, cuento la raíz como 1 y sumo la cantidad de nodos del subárbol izquierdo y del subárbol derecho.

```
public static int contarNodos(ABBTDA a) {
    if (a.ArbolVacio()) {
        return 0;
    } else {
        return 1 + contarNodos(a.HijoIzq()) +
        contarNodos(a.HijoDer());
    }
}
```

Ejemplo:



Cantidad total de nodos: 9.

La raíz 50 cuenta como 1. Luego se cuentan los nodos del subárbol izquierdo, que son 30, 20, 40, 35, 45, y los del subárbol derecho, que son 70, 60, 80.

4. Contar hojas

Una **hoja** es un nodo que no tiene hijos. Es decir, su subárbol izquierdo está vacío y su subárbol derecho también está vacío.

En el árbol anterior, las hojas son:

20, 35, 45, 60, 80

El método para contar hojas tiene tres situaciones:

- Si el árbol está vacío, devuelve 0.
- Si el árbol no está vacío y no tiene hijos, devuelve 1.
- Si tiene hijos, suma las hojas del subárbol izquierdo y derecho.

```
public static int contarHojas(ABBTDA a) {
    if (a.ArbolVacio()) {
        return 0;
    } else if (a.HijoIzq().ArbolVacio() && a.HijoDer().ArbolVacio()) {
```

```

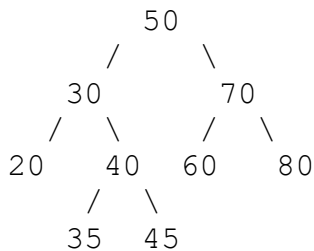
    return 1;
  } else {
    return contarHojas(a.Hijolq()) + contarHojas(a.HijoDer());
  }
}

```

Este método es importante porque obliga a distinguir entre **árbol vacío**, **nodo hoja** y **nodo interno**. Un error común es pensar que cualquier nodo de abajo es hoja, pero lo correcto es verificar si tiene o no tiene hijos.

5. Contar nodos internos

Un **nodo interno** es un nodo que tiene al menos un hijo. Por lo tanto, no es una hoja. En el ejemplo:



Los nodos internos son:

50, 30, 40, 70

El método puede resolverse así:

```

public static int contarInternos(ABBTDA a) {
  if (a.ArbolVacio()) {
    return 0;
  } else if (a.Hijolq().ArbolVacio() && a.HijoDer().ArbolVacio()) {
    return 0;
  } else {
    return 1 + contarInternos(a.Hijolq()) + contarInternos(a.HijoDer());
  }
}

```

La lógica es parecida a contar hojas, pero al revés: si el nodo no tiene hijos, no se cuenta como interno. Si tiene al menos un hijo, se cuenta como 1 y luego se siguen recorriendo sus subárboles.

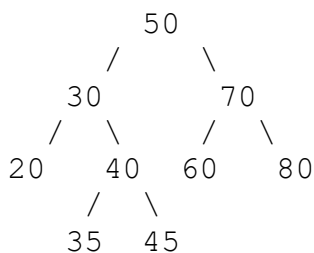
6. Calcular la altura de un ABB

La **altura** de un árbol es la mayor distancia desde la raíz hasta una hoja. Si usamos el criterio de que la raíz tiene profundidad 0, entonces un árbol con un solo nodo tiene altura 0, y un árbol vacío puede considerarse con altura -1.

Este criterio es muy útil porque permite que la fórmula recursiva sea clara:

```
public static int altura(ABBTDA a) {
    if (a.ArbolVacio()) {
        return -1;
    } else {
        int alturalzq = altura(a.Hijolzq());
        int alturaDer = altura(a.HijoDer());
        return 1 + Math.max(alturalzq, alturaDer);
    }
}
```

Ejemplo:



El camino más largo desde la raíz hasta una hoja puede ser:

50 → 30 → 40 → 35

o

50 → 30 → 40 → 45

Ese camino tiene 3 aristas. Por lo tanto, la altura del árbol es 3.

Es importante aclarar el criterio, porque algunos materiales definen la altura contando nodos y otros contando aristas. En esta clase usaremos el criterio de profundidad desde raíz 0, por lo tanto la altura se mide por cantidad de aristas hasta la hoja más profunda.

7. Calcular la profundidad de un nodo

La **profundidad** de un nodo es la distancia desde la raíz hasta ese nodo. La raíz tiene profundidad 0. Sus hijos tienen profundidad 1. Los hijos de esos hijos tienen profundidad 2, y así sucesivamente.

Para calcular la profundidad de un valor x , conviene aprovechar la regla del ABB. Si x es menor que la raíz, se busca por izquierda. Si es mayor, por derecha. Si se encuentra, se devuelve 0 desde ese punto y se suma 1 por cada retorno.

```
public static int profundidad(ABBTDA a, int x) {
    if (a.ArbolVacio()) {
        return -1;
    } else if (a.Raiz() == x) {
        return 0;
    } else if (x < a.Raiz()) {
        int p = profundidad(a.Hijolq(), x);
        if (p == -1) {
            return -1;
        } else {
            return 1 + p;
        }
    } else {
        int p = profundidad(a.HijoDer(), x);
        if (p == -1) {
            return -1;
        } else {
            return 1 + p;
        }
    }
}
```

Ejemplo: profundidad de 45.

50 → 30 → 40 → 45

La profundidad es 3.

Si el valor no existe, el método devuelve -1.

8. Buscar un elemento

La búsqueda en ABB no necesita recorrer todo el árbol. Usa la propiedad de orden.

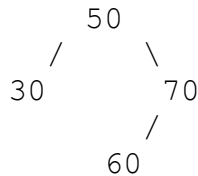
```
public static boolean existe(ABBTDA a, int x) {
    if (a.ArbolVacio()) {
        return false;
    } else if (a.Raiz() == x) {
        return true;
    } else if (x < a.Raiz()) {
        return existe(a.Hijolq(), x);
    }
}
```

```

    } else {
        return existe(a.HijoDer(), x);
    }
}

```

Ejemplo: buscar 60.



Primero se compara 60 con 50. Como $60 > 50$, se baja por derecha. Luego se compara 60 con 70. Como $60 < 70$, se baja por izquierda. Finalmente se encuentra 60.

El camino recorrido es:

$50 \rightarrow 70 \rightarrow 60$

Si se buscara 90, se recorrería:

$50 \rightarrow 70 \rightarrow 80 \rightarrow \text{vacío}$

Al llegar a un árbol vacío, se devuelve false.

9. Obtener el menor valor

En un ABB, el menor valor siempre se encuentra bajando hacia la izquierda. Esto ocurre porque todos los valores menores se ubican en el subárbol izquierdo.

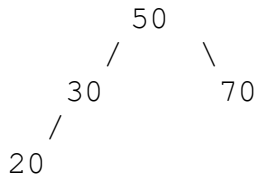
```

public static int menor(ABBTDA a) {
    if (a.Hijolq().ArbolVacio()) {
        return a.Raiz();
    } else {
        return menor(a.Hijolq());
    }
}

```

Precondición: el árbol debe estar inicializado y no vacío.

Ejemplo:



El menor es 20.

El método baja desde 50 a 30, y de 30 a 20. Como 20 no tiene hijo izquierdo, se devuelve 20.

10. Obtener el mayor valor

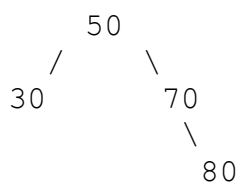
En un ABB, el mayor valor siempre se encuentra bajando hacia la derecha.

```

public static int mayor(ABBTDA a) {
    if (a.HijoDer().ArbolVacio()) {
        return a.Raiz();
    } else {
        return mayor(a.HijoDer());
    }
}
  
```

Precondición: el árbol debe estar inicializado y no vacío.

Ejemplo:



El mayor es 80.

El método baja desde 50 a 70, y luego de 70 a 80. Como 80 no tiene hijo derecho, se devuelve 80.

11. Mostrar valores pares

Otro tipo de problema frecuente consiste en recorrer todo el árbol y aplicar una condición. Por ejemplo, mostrar solamente los valores pares.

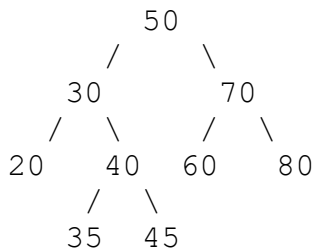

```
public static void mostrarPares(ABBTDA a) {
    if (!a.ArbolVacio()) {
        mostrarPares(a.Hijolq());

        if (a.Raiz() % 2 == 0) {
            System.out.print(a.Raiz() + " ");
        }

        mostrarPares(a.HijoDer());
    }
}
```

En este caso usamos recorrido inorden: izquierda, raíz, derecha. Como el árbol es un ABB, los pares se muestran ordenados de menor a mayor.

Para el árbol:



La salida sería:

20 30 40 50 60 70 80

Todos son pares excepto 35 y 45.

12. Sumar todos los valores

Para sumar todos los valores del árbol, el caso base es claro: si el árbol está vacío, suma 0. Si no está vacío, se suma la raíz más la suma del subárbol izquierdo y la del subárbol derecho.

```
public static int sumar(ABBTDA a) {
    if (a.ArbolVacio()) {
        return 0;
    } else {
        return a.Raiz() + sumar(a.Hijolq()) + sumar(a.HijoDer());
    }
}
```

Aplicado al árbol de ejemplo:

$50 + 30 + 70 + 20 + 40 + 60 + 80 + 35 + 45 = 430$

Resultado: 430.

13. Sumar valores mayores que X

Este problema permite aprovechar la regla del ABB para descartar ramas. Si la raíz es menor o igual que x, entonces todo el subárbol izquierdo también será menor o igual que la raíz, por lo que no hace falta recorrerlo. En ese caso, solo interesa el subárbol derecho.

```
public static int sumarMayoresQue(ABBTDA a, int x) {
    if (a.ArbolVacio()) {
        return 0;
    } else if (a.Raiz() <= x) {
        return sumarMayoresQue(a.HijoDer(), x);
    } else {
        return a.Raiz()
            + sumarMayoresQue(a.HijoIzq(), x)
            + sumarMayoresQue(a.HijoDer(), x);
    }
}
```

Ejemplo: sumar mayores que 45.

Valores mayores que 45:

50, 60, 70, 80

Resultado:

260

Este método es interesante porque no siempre recorre todo el árbol. Usa la propiedad del ABB para evitar trabajo innecesario.

14. Listar valores dentro de un rango

Otro problema aplicado es listar los valores entre dos límites, por ejemplo entre desde y hasta.

```
public static void listarRango(ABBTDA a, int desde, int hasta) {
    if (!a.ArbolVacio()) {
        if (a.Raiz() > desde) {
```

```

    listarRango(a.Hijolq(), desde, hasta);
}

if (a.Raiz() >= desde && a.Raiz() <= hasta) {
    System.out.print(a.Raiz() + " ");
}

if (a.Raiz() < hasta) {
    listarRango(a.HijoDer(), desde, hasta);
}
}
}

```

Ejemplo: listar valores entre 35 y 70.

Salida esperada: 35 40 45 50 60 70

Este ejercicio combina tres ideas: recorrido inorden, condición y poda de ramas. Si la raíz es menor que el límite inferior, no necesito ir a la izquierda. Si la raíz es mayor que el límite superior, no necesito ir a la derecha.

15. Balance conceptual

Un ABB puede ser correcto y aun así tener mala forma. La propiedad de orden puede cumplirse, pero el árbol puede quedar muy inclinado hacia un lado.

ABB balanceado:

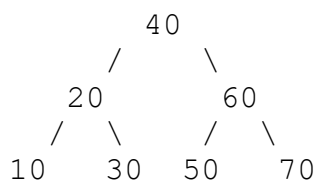
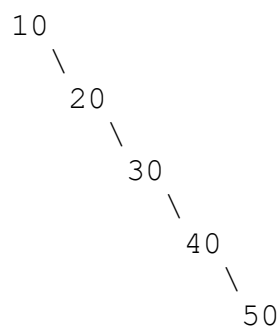


ABB desbalanceado:



Los dos pueden cumplir la regla de ABB, pero no tienen el mismo rendimiento. En el primer caso, la altura es baja. En el segundo, el árbol se parece a una lista enlazada.

Este problema es importante porque la eficiencia del ABB depende de su altura. Si el árbol está bien distribuido, la búsqueda puede acercarse a $O(\log n)$. Si el árbol queda totalmente desbalanceado, la búsqueda puede acercarse a $O(n)$. Este punto prepara el estudio de AVL, que agrega una condición de equilibrio para evitar que el ABB se degrade hacia una estructura similar a una lista.

16. Detectar si un nodo está desbalanceado

Para esta clase vamos a usar una idea conceptual de desbalance. Diremos que un nodo está desbalanceado si la diferencia entre la altura de su subárbol izquierdo y la altura de su subárbol derecho es mayor que 1 en valor absoluto.

Es decir:

$$|\text{altura izquierda} - \text{altura derecha}| > 1$$

Método auxiliar:

```
public static boolean estaDesbalanceado(ABBTDA a) {
    if (a.ArbolVacio()) {
        return false;
    }

    int altIzq = altura(a.Hijolq());
    int altDer = altura(a.HijoDer());

    return Math.abs(altIzq - altDer) > 1;
}
```

Este método no corrige el árbol. Solo detecta si un nodo está desbalanceado.

17. Listar nodos desbalanceados

Ahora podemos recorrer todo el árbol y mostrar aquellos nodos que están desbalanceados.

```
public static void listarDesbalanceados(ABBTDA a) {
    if (!a.ArbolVacio()) {
        listarDesbalanceados(a.Hijolq());

        if (estaDesbalanceado(a)) {
```

```

        System.out.print(a.Raiz() + " ");
    }

    listarDesbalanceados(a.HijoDer());
}
}

```

Este método usa modularidad:

- altura(a) calcula la altura de un árbol.
- estaDesbalanceado(a) evalúa un nodo.
- listarDesbalanceados(a) recorre todo el árbol.

Esta forma de dividir el problema es importante. En ejercicios más complejos, no conviene resolver todo en un único método largo. Es mejor construir pequeñas funciones auxiliares con responsabilidades claras.

18. Análisis de costo

El costo de las operaciones en un ABB depende de la altura del árbol.

En una búsqueda, inserción o eliminación, normalmente se baja por un solo camino desde la raíz hasta una hoja. Por eso el costo puede expresarse como:

$O(h)$

donde h es la altura del árbol.

Si el árbol está balanceado, la altura se aproxima a $\log n$, por lo que el costo se acerca a:

$O(\log n)$

Si el árbol está desbalanceado, la altura puede acercarse a n , por lo que el costo se vuelve:

$O(n)$

En cambio, los métodos que recorren todo el árbol, como contarNodos, sumar, mostrarPares o contarHojas, tienen costo:

$O(n)$

porque visitan todos los nodos una vez.

Hay que distinguir entonces dos tipos de problemas:

- **Problemas de búsqueda dirigida**, donde se aprovecha la regla ABB y se baja por una rama.
- **Problemas de recorrido completo**, donde se necesita visitar todo el árbol.

19. Estrategia para resolver ejercicios

Antes de programar, conviene completar mentalmente esta guía:

1. ¿El método devuelve algo o solo muestra datos?
2. ¿Cuál es el caso base si el árbol está vacío?
3. ¿Debo procesar la raíz?
4. ¿Debo recorrer ambos hijos o solo uno?
5. ¿Puedo aprovechar la propiedad del ABB para descartar ramas?
6. ¿Necesito un método auxiliar?
7. ¿Cuál es el costo esperado?

Ejemplo: contar hojas.

1. Devuelve un entero.
2. Si está vacío, devuelve 0.
3. Si la raíz no tiene hijos, cuenta como hoja.
4. Si tiene hijos, recorre ambos subárboles.
5. No se aprovecha orden ABB porque hay que revisar la estructura.
6. No necesita auxiliar.
7. Costo $O(n)$.

Ejemplo: buscar un valor.

1. Devuelve boolean.
2. Si está vacío, devuelve false.
3. Si la raíz es el valor, devuelve true.
4. Si no, baja solo por izquierda o derecha.
5. Sí aprovecha la propiedad del ABB.
6. No necesita auxiliar.
7. Costo $O(h)$: $O(\log n)$ si está balanceado, $O(n)$ si está desbalanceado.

20. Actividad guiada de clase

Dado el ABB construido con la siguiente secuencia:

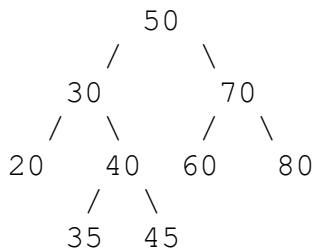
50, 30, 70, 20, 40, 60, 80, 35, 45

Resolver en grupos:

1. Dibujar el ABB resultante.
2. Indicar raíz, hojas, nodos internos, altura y profundidad de 45.
3. Implementar contarNodos.
4. Implementar contarHojas.
5. Implementar altura.
6. Implementar menor.
7. Implementar mayor.
8. Implementar mostrarPares.
9. Implementar sumar.
10. Implementar listarDesbalanceados.

21. Respuestas esperadas sobre el árbol base

Árbol:



Raíz:

50

Hojas:

20, 35, 45, 60, 80

Nodos internos:

50, 30, 40, 70

Altura:

3

Profundidad de 45:

3

Cantidad de nodos:

9

Cantidad de hojas:

5

Menor:

20

Mayor:

80

Suma total:

430

Pares en orden:

20 30 40 50 60 70 80

Resolver problemas con ABB no consiste en memorizar métodos aislados. La clave está en reconocer la estructura recursiva del árbol y decidir qué hacer en cada caso. Si el árbol está vacío, se aplica el caso base. Si no está vacío, se analiza la raíz y se decide si corresponde recorrer el hijo izquierdo, el derecho o ambos.

Cuando el problema exige procesar todos los nodos, el costo suele ser $O(n)$. Cuando el problema aprovecha la propiedad de búsqueda del ABB, el costo depende de la altura: puede ser eficiente si el árbol está balanceado, pero puede degradarse si el árbol queda inclinado como una lista. Esa dificultad abre el camino hacia la próxima clase: los árboles AVL, que nacen justamente para controlar el desbalance y mantener la búsqueda eficiente.